Induction and Recursion

Shuai Zhang

Department of Computer Science
University of Central Oklahoma

Introduction

- On the topic of Sequences, we learned "**recurrence relation**": $f_n = f_{n-1} + f_{n-2} + \cdots + f_1$
  - Recurrence relation + initial condition = closed formula
    - We can find all the terms in the sequence
- If we want to show predicate $P(n)$ is true for any positive/non-negative integer n
  - We can prove
    - The base case $P(k_0)$: k_0 is usually the smallest valid value
    - Recursive induction: $P(k) \to P(k+1)$ for any valid k
      - $P(1) \to P(2) \to P(3) \to \cdots \to P(n-1) \to P(n)$: **block dominos**!
  - Then $P(n)$ is true for any n

Objectives and Summary

- Recursive Definitions
  - (1) $P(1)$
  - (2) $P(k) \rightarrow P(k+1)$
  - Keep applying (2), we can say:
    - $P(1) \rightarrow P(2)$, $P(2) \rightarrow P(3)$, $P(3) \rightarrow P(4)$, …
    - $P(1) \rightarrow P(2) \rightarrow P(3) \rightarrow \cdots \rightarrow P(k) \rightarrow P(k+1)$
    - $P(n)$ can be calculated
- Recursive Algorithms
  - $P(n) \rightarrow P(n-1) \rightarrow P(n-2) \rightarrow \cdots \rightarrow P(2) \rightarrow P(1)$
- Mathematical Induction
  - True or false version of in recursive definition
  - (1) $P(1)$ is true
  - (2) $P(k) \rightarrow P(k+1)$
  - Keep applying (2), we can say:
    - $P(1) \rightarrow P(2)$, $P(2) \rightarrow P(3)$, $P(3) \rightarrow P(4)$, …
    - $P(1) \rightarrow P(2) \rightarrow P(3) \rightarrow \cdots \rightarrow P(k) \rightarrow P(k+1)$
    - $P(n)$ is true

Recursive Definitions

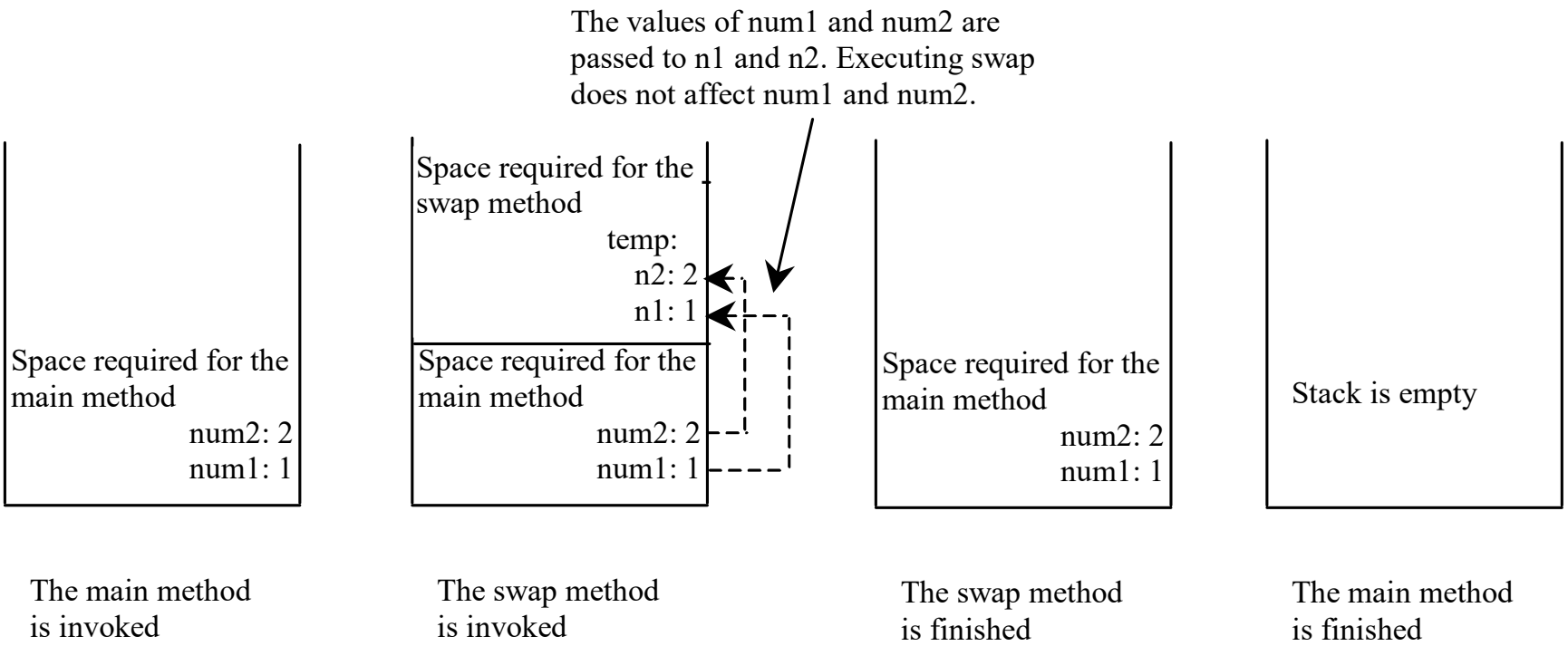
- Recursive definition is also called inductive definition
- Two steps:
  - **Basis Step**: Specify the initial values or elements
  - **Recursive Step**: Give the rule for finding other values or elements one by one in a chain
- Example:
  - Recursive definition for a function:
    - Basis step: $f(0) = 3$
    - Recursive step: $f(n+1) = 2f(n) + 3$
      - $f(1) = 9, f(2) = 21, f(3) = 45, f(4) = 93 \ldots$
      - Keep using previous values or elements to find others
- Recursive definitions are well defined: unambiguous

Recursive Definitions: More Examples

- E1: recursively define a^n
  - Basis step: a^0
  - Recursive step: $a^n = a \cdot a^{n-1}$
- E2: recursively define $\sum_{k=0}^{n} a_k$
  - Basis step: $\sum_{k=0}^{0} a_k = a_0$
  - Recursive step: $\sum_{k=0}^{n+1} a_k = \sum_{k=0}^{n} a_k + a_{n+1}$
- E3: recursively define Fibonacci numbers
  - Basis step: $f_0 = 0$, $f_1 = 1$
  - Recursive step: $f_n = f_{n-1} + f_{n-2}$
- E4: recursively define a set $\{x = 3k | k \in \mathbb{Z}^+\}$
  - Basis step: $3 \in S$
  - Recursive step: if $x \in S$ and $y \in S$, then $x + y \in S$

Recursive Algorithm 1

- How recursion is supported 1
  - Call stack + activation records
  - Main function calls swap function

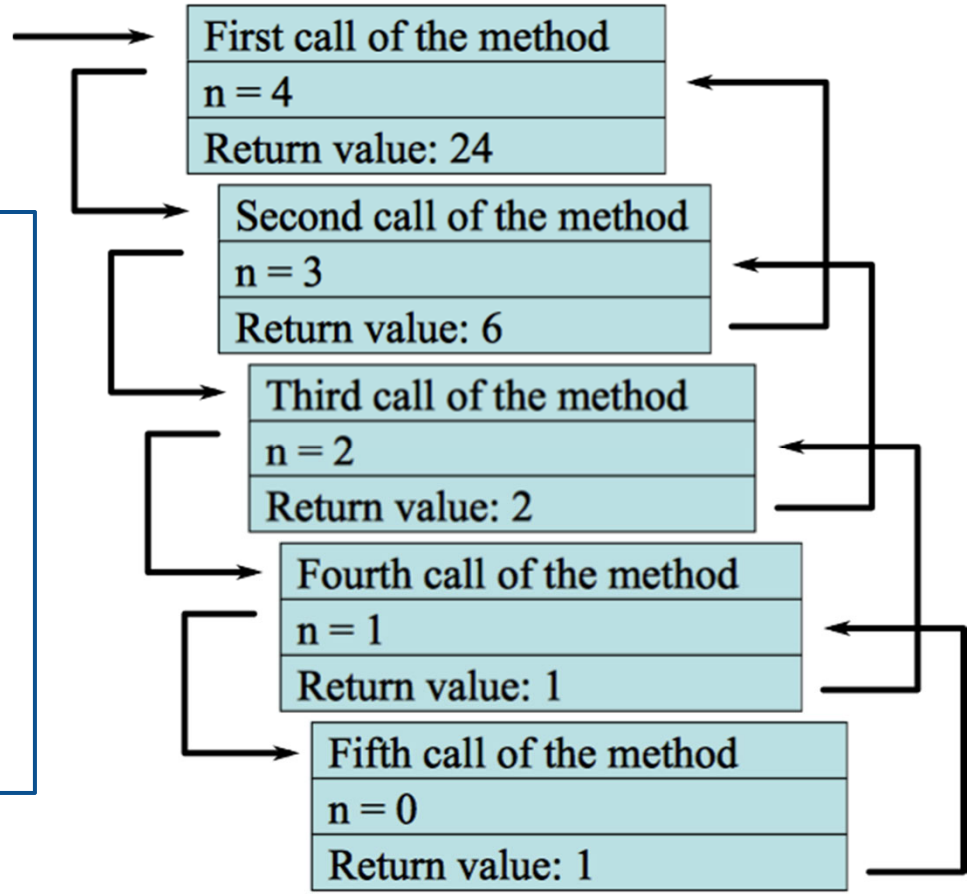


Recursive Algorithm 2

- How recursion is supported 2

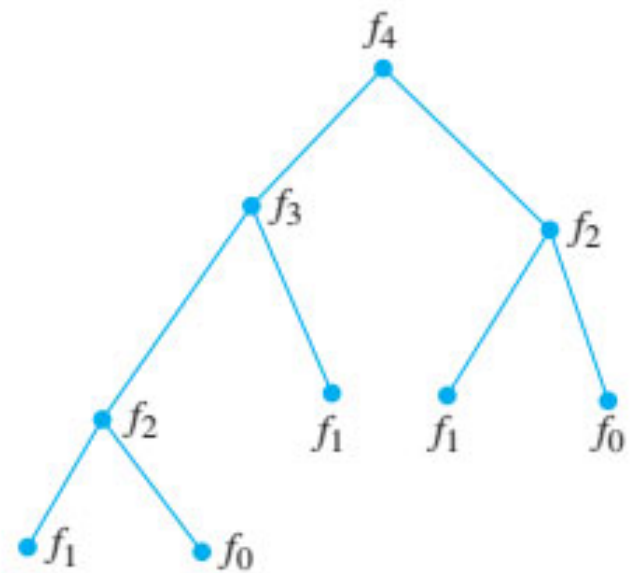
$$n! = n \cdot (n-1)!$$

procedure *factorial*(*n*:
nonnegative integer)
if *n* = 0 **then return** 1
else
return *n* · *factorial*(*n* - 1)
{output is *n*!}



Recursive Algorithm 3

- In recursive definition, the procedure starts from the basis step (base case) ---- the forward way
- Can we think in a backward way?
  - e.g. in Fibonacci sequence, if we want to know f_n, we need f_{n-1} and f_{n-2}. And we trace back to the basis step: f_0 and f_1
    - Visualized procedure as a tree
    - Keys:
      - Recurrence relation
      - Initial terms



Recursive Algorithm 4

- Think in the backward way for a program
  - The calculation procedures for f_{n-1} and f_{n-2} are exactly the same as the calculation of f_n. We can repeat the same step until we get f_0 and f_1

```
procedure fibonacci(n: nonnegative integer)
  if n = 0 then return 0
  else if n = 1 then return 1
  else return fibonacci(n - 1) + fibonacci(n - 2)
  {output is fibonacci(n)}
```

- An algorithm is called **recursive** if it solves a problem by reducing it to an instance of the same problem with smaller input -- **subproblems**

Examples of Recursive Algorithms$_1$

- **E1**: Give a recursive algorithm for computing $n!$
- **Solution**: $n! = n \cdot (n-1)!$

```
procedure factorial(n: nonnegative integer)
  if n = 0 then return 1
  else return n · factorial(n - 1)
  {output is n!}
```

- **E2**: Give a recursive algorithm for computing a^n
- **Solution**: $a^n = a \cdot a^{n-1}$

```
procedure power(a: nonzero real number, n:
nonnegative integer)
  if n = 0 then return 1
  else return a · power(a, n - 1)  {output is a^n}
```

Examples of Recursive Algorithms$_2$

- **E3**: Give a recursive algorithm for computing the greatest common divisor of two non-negative integers a and b with $a < b$
- **Solution**: $gcd(a, b) = gcd(b \, \boldsymbol{mod} \, a, a)$

```
procedure gcd(a, b: nonnegative integers with a < b)
    if a = 0 then return b
    else return gcd(b mod a, a)
    {output is gcd(a, b)}
```

- Sometimes recursive algorithms are much easier comparing to the forward algorithm
- In human logic, is this really backward or forward?

Formal Definition: Principle of Mathematical Induction

- To prove that $P(n)$ is true for all positive integers *n*, we complete these steps:
  - ***Basis Step***: Show that $P(k_0)$ is true.
  - ***Inductive Step***: Show that $P(k) \rightarrow P(k+1)$ is true for all positive integers k.
- In inductive step, $P(k)$ is an assumption, we assume $P(k)$ is true, and further prove $P(k+1)$ is true
  - $P(k)$ is called **inductive hypothesis**
  - **Remark**: this assumption is **only used to show the inductive relation**, not to show $P(k)$ is always true

Mathematical Induction Examples 1

- **Prove**: $1 + 2 + .. + n = \frac{n(n+1)}{2}$
- **Solution**: (1). **Basis step**: $P(1) = 1 = \frac{1(1+1)}{2}$

  (2). **Inductive step**: $P(k) \rightarrow P(k+1)$ for any k

  Inductive hypothesis: $P(k) = 1 + 2 + .. + k = \frac{k(k+1)}{2}$

$$1 + 2 + .. + k + (k+1) = \frac{k(k+1)}{2} + (k+1)$$

$$= (k+1)\left(\frac{k}{2} + 1\right) = (k+1)\frac{k+2}{2}$$

$$= \frac{(k+1)(k+1+1)}{2} = P(k+1)$$

Mathematical Induction Examples 2

- **Prove**: $n < 2^n$ for all positive integers n
- **Solution**: (1). **Basis step**: $P(1)$: $1 < 2$
  (2). **Inductive step**:
  Inductive hypothesis: $P(k)$: $k < 2^k$, then
  $k + 1 < 2^k + 1 < 2^k + 2^k = 2^{k+1}$
  $\rightarrow P(k+1)$
- **Prove**: $2^n < n!$ for all positive integers $n \geq 4$
- **Solution**: (1). **Basis step**: $P(4)$: $2^4 = 16 < 4! = 24$
  (2). **Inductive step**: **Inductive hypothesis**:
  $P(k)$: $2^k < k!$, then
  $2^{k+1} = 2 \cdot 2^k < 2 \cdot k! < (k+1) \cdot k! = (k+1)!$
  $\rightarrow P(k+1)$

Mathematical Induction Examples 3

- **Prove**: $n^3 - n$ is divisible by 3, for any positive integers n
- **Solution**: (1). **Basis step**: $P(1)$: $3|1 - 1 = 0$
  (2). **Inductive step**:
  Inductive hypothesis $P(k)$: $3|k^3 - k$, then
  $(k+1)^3 - (k+1) = k^3 + 3k^2 + 3k + 1 - (k+1)$
  $= (k^3 - k) + 3(k^2 + k)$ is divisible by 3 $\rightarrow$
  $P(k+1)$

Second Thought on Recurrence Relation

- On the topic of Sequences, we learned "**recurrence relation**": $f_n = f_{n-1} + f_{n-2} + \cdots + f_1$
  - Recurrence relation + initial condition = closed formula
    - We can find all the terms in the sequence
- Strong Induction (or complete induction):
  - ***Basis Step***: Show that $P(k_0)$ is true.
  - ***Inductive Step***: Show that $(P(1) \wedge P(2) \wedge \cdots \wedge P(k)) \rightarrow P(k+1)$ is true for all positive integers k.

$\boldsymbol{P(1)}$				
$\boldsymbol{P(1)} \rightarrow$	$\boldsymbol{P(2)}$			
$\boldsymbol{P(1)}$	$\boldsymbol{P(2)} \rightarrow$	$\boldsymbol{P(3)}$		
$\boldsymbol{P(1)}$	$\boldsymbol{P(2)}$	$\boldsymbol{P(3)} \rightarrow$	$\boldsymbol{P(4)}$	
…	…	…	… $\rightarrow$	…

By this chain effect we can get $P(n)$ is true for any positive integer n.

Example of Strong Induction

- **Prove**: Every amount of postage of 12 cents or more can be formed using just 4-cent and 5-cent stamps.
- **Solution**: (1). **Basis step**: $P(12): 12 = 4 + 4 + 4$, $P(13)$: $13 = 4 + 4 + 5$, $P(14)$: $14 = 4 + 5 + 5$, $P(15)$: $15 = 5 + 5 + 5$ Why they are needed?

  (2).**Inductive step**:
  Inductive hypothesis: $P(j)$ is true for $12 \leq j \leq k$ where $k \geq 15$. Then for $P(k+1)$ is can be formed by adding a 4-cent stamp to $\boxed{P(k-3)}$, which is one of the case in the hypothesis, because $12 \leq k - 3 \leq k$

Practice 1

- 1. Determine whether the proposed definitions are valid recursive definitions of functions from nonnegative integers to integers.
  - $f(0)=0$, $f(n)=2f(n-2)$, for $n \geq 1$
  - $f(0)=1$, $f(n)=3f(n-1)$ if n is odd and $n \geq 1$, $f(n)=9f(n-2)$ if n is even and $n \geq 2$

Practice 2

- 1. Determine whether the proposed definitions are valid recursive definitions of functions from nonnegative integers to integers.
  - $f(0)=0$, $f(n)=2f(n-2)$, for $n \geq 1$
    - No. $f(1)$ is not defined.
  - $f(0)=1$, $f(n)=3f(n-1)$ if n is odd and $n \geq 1$, $f(n)=9f(n-2)$ if n is even and $n \geq 2$
    - Yes.
    - $f(1)=3f(0)=3$,
    - $f(2)=9f(0)=9$,
    - $f(3)=3f(2)=27$,
    - $f(4)=9f(2)=81$,… $f(n)=3^n$, for $n \geq 0$

Practice 3

- 2. Do you agree with gcd(a,b)=gcd(a, b-a) if a<b? If so, please give a recursive algorithm to calculate gcd(a,b) when a<b.

Practice 4

- 2. Do you agree with gcd(a,b)=gcd(a, b-a) if a<b? If so, please give a recursive algorithm to calculate gcd(a,b) when a<b.
  - Yes: b = a + (b-a), so b-a also share the common divisors, including gcd

```
procedure gcd(a,b: nonnegative integers)
  if a=0 then return b
  if a = b-a then return a
  if a < b-a then return gcd(a, b-a)
  geturn gcd(b-a, a)
```

Practice 5

- 3. Prove: for any nonnegative integer n, $1^2+3^2+5^2+\ldots+(2n+1)^2 = (n+1)(2n+1)(2n+3)/3$

Practice 6

- 3. Prove: for any nonnegative integer n, $1^2+3^2+5^2+\ldots+(2n+1)^2 = (n+1)(2n+1)(2n+3)/3$
  - Mathematic induction
    - (1) Basic step - when n is 1: $1+3^2 = (2*3*5)/3=10$
    - (2) Inductive hypothesis: for any nonnegative integer k, we assume $1^2+3^2+5^2+\ldots+(2k+1)^2 = (k+1)(2k+1)(2k+3)/3$
    - (3) Based on this hypothesis let's prove it works k+1, where the target is
      $1^2+3^2+5^2+\ldots+(2k+3)^2 = (k+2)(2k+3)(2k+5)/3$
    - Below are optional:
      $1^2+3^2+5^2+\ldots+(2k+3)^2 = 1^2+3^2+5^2+\ldots(2k+1)^2 +(2k+3)^2$
      $= (k+1)(2k+1)(2k+3)/3+(2k+3)^2$
      $=(2k+3)((k+1)(2k+1)+3(2k+3))/3$
      $=(2k+3)(2k^2+9k+10)/3$
      $=(2k+3)(k+2)(2k+5)/3$
      $= (k+2)(2k+3)(2k+5)/3$

Extension

- Strong induction and mathematic induction are equivalent
  - Strong -> Weak: If you can prove a statement by assuming all previous steps are true (strong), you can certainly prove it by assuming only the last step is true (weak).
  - Weak -> Strong: You can convert any strong induction proof into a weak induction proof by redefining the proposition. Instead of proving P(n), you prove a new statement Q(n), defined as "P(1) and P(2) and ... and P(n) are all true."